

尚硅谷大数据技术之 Spark 内核

(作者: 尚硅谷研究院)

版本: V5.0

第 1 章 环境准备及提交流程

1.1 程序起点

1) spark-3.3.1-bin-hadoop3\bin\spark-submit.cmd

=> cmd /V /E /C ""%~dp0spark-submit2.cmd" %*"

2) spark-submit2.cmd

=> set CLASS=org.apache.spark.deploy.SparkSubmit

"%~dp0spark-class2.cmd" %CLASS% %*

3) spark-class2.cmd

=> %SPARK_CMD%

4) 在 spark-class2.cmd 文件中增加打印%SPARK_CMD%语句

```
echo %SPARK_CMD%  
%SPARK_CMD%
```

5) 在 spark-3.3.1-bin-hadoop3\bin 目录上执行 cmd 命令

6) 进入命令行窗口, 输入

```
bin/spark-submit --class org.apache.spark.examples.SparkPi --master  
local[2] ./examples/jars/spark-examples_2.12-3.3.1.jar 10
```

```
E:\02_software\spark-3.0.0-bin-hadoop3.2\bin>spark-submit --class org.apache.spa  
rk.examples.SparkPi --master local[2] ../examples/jars/spark-examples_2.12-3.0.0  
.jar 10  
Missing Python executable 'python', defaulting to 'E:\02_software\spark-3.0.0-bi  
n-hadoop3.2\bin\..' for SPARK_HOME environment variable. Please install Python o  
r specify the correct Python executable in PYSPARK_DRIVER_PYTHON or PYSPARK_PYTH  
ON environment variable to detect SPARK_HOME safely.  
E:\02_software\jdk1.8.0_144\bin\java -cp "E:\02_software\spark-3.0.0-bin-ha  
doo3.2\bin\..\conf\;E:\02_software\spark-3.0.0-bin-hadoop3.2\bin\..\jars\*" -Xm  
x1g org.apache.spark.deploy.SparkSubmit --master local[2] --class org.apache.spa  
rk.examples.SparkPi ../examples/jars/spark-examples_2.12-3.0.0.jar 10
```

7) 发现底层执行的命令为

```
java -cp org.apache.spark.deploy.SparkSubmit
```

说明: java -cp 和 -classpath 一样, 是指定类运行所依赖其他类的路径。

8) 执行 java -cp 就会开启 JVM 虚拟机, 在虚拟机上开启 SparkSubmit 进程, 然后开始执行 main 方法

java -cp =》开启 JVM 虚拟机 =》开启 Process (SparkSubmit) =》程序入口

更多 Java - 大数据 - 前端 - python 人工智能资料下载, 可百度访问: 尚硅谷官网

SparkSubmit.main

9) 在 IDEA 中全局查找 (ctrl + n): `org.apache.spark.deploy.SparkSubmit`, 找到 `SparkSubmit` 的伴生对象, 并找到 `main` 方法

```
override def main(args: Array[String]): Unit = {
  val submit = new SparkSubmit() {
    ... ..
  }
}
```

1.2 Spark 组件通信

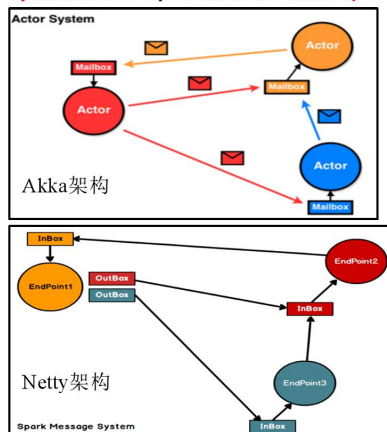
1.2.1 Spark 中通信框架的发展



Spark中通信框架的发展



- Spark早期版本中采用Akka作为内部通信部件。
- Spark1.3中引入Netty通信框架, 为了解决Shuffle的大数据传输问题使用
- Spark1.6中Akka和Netty可以配置使用。Netty完全实现了Akka在Spark中的功能。
- Spark2.x系列中, Spark抛弃Akka, 使用Netty。

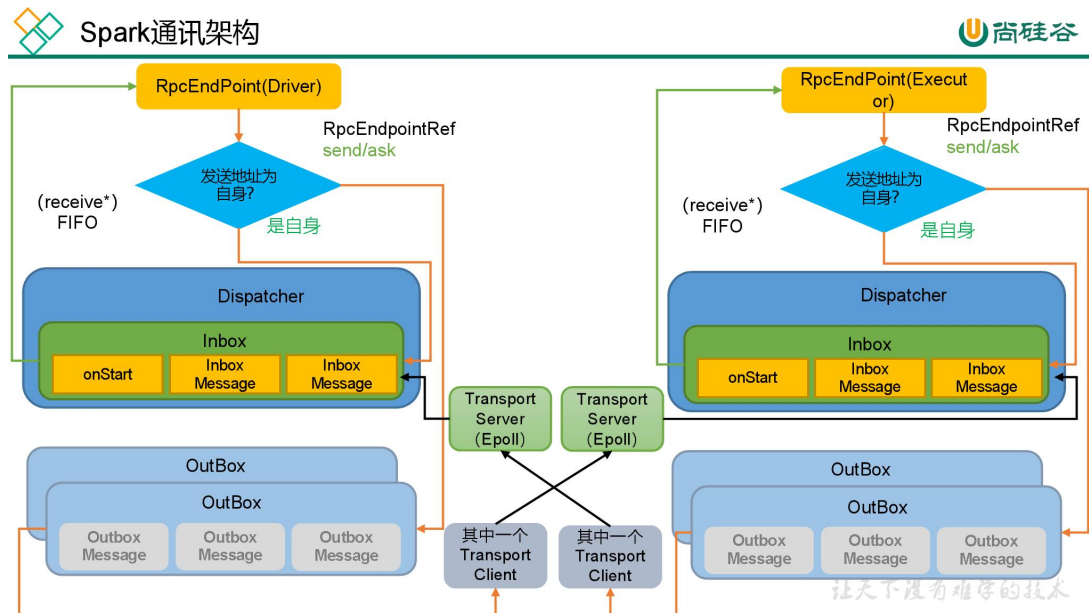


那么Netty为什么可以取代Akka? 首先不容置疑的是Akka可以做到的, Netty也可以做到, 但是Netty可以做到, Akka却无法做到, 原因是啥? 在软件栈中, Akka相比Netty要Higher一点, 它专门针对RPC做了很多事情, 而Netty相比更加基础一点, 可以为不同的应用层通信协议 (RPC, FTP, HTTP等) 提供支持, 在早期的Akka版本, 底层的NIO通信就是用的Netty; 其次一个优雅的工程是不会允许一个系统中容纳两套通信框架, 恶心! 最后, 虽然Netty没有Akka协程级的性能优势, 但是Netty内部高效的Reactor线程模型, 无锁化的串行设计, 高效的序列化, 零拷贝, 内存池等特性也保证了Netty不会存在性能问题。

Endpoint有1个InBox和N个OutBox (N>=1, N取决于当前Endpoint与多少其他的Endpoint进行通信, 一个与其通讯的其他Endpoint对应一个OutBox), Endpoint接收到的消息被写入InBox, 发送出去的消息写入OutBox并被发送到其他Endpoint的InBox中。

Spark 底层采用 netty 通信。

1.2.2 Spark 底层通信原理



- **RpcEndpoint: RPC 通信终端。**Spark 针对每个节点（Client/Master/Worker）都称之为一个 RPC 终端，且都实现 RpcEndpoint 接口，内部根据不同端点的需求，设计不同的消息和不同的业务处理，如果需要发送（询问）则调用 Dispatcher。在 Spark 中，所有的终端都存在生命周期：

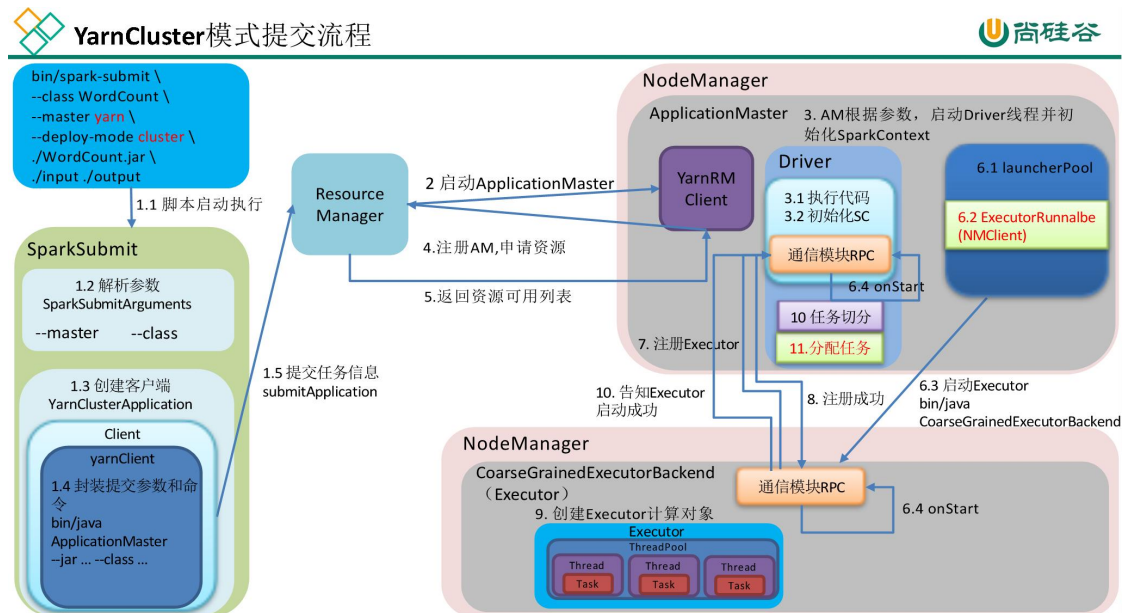
Constructor =» onStart =» receive* =» onStop

- **RpcEnv:** RPC 上下文环境，每个 RPC 终端运行时依赖的上下文环境称为 RpcEnv；在当前 Spark 版本中使用的 NettyRpcEnv
- **Dispatcher:** 消息调度（分发）器，针对于 RPC 终端需要发送远程消息或者从远程 RPC 接收到的消息，分发至对应的指令收件箱（发件箱）。如果指令接收方是自己则存入收件箱，如果指令接收方不是自己，则放入发件箱；
- **Inbox:** 指令消息收件箱。一个本地 RpcEndpoint 对应一个收件箱，Dispatcher 在每次向 Inbox 存入消息时，都将对应 EndpointData 加入内部 ReceiverQueue 中，另外 Dispatcher 创建时会启动一个单独线程进行轮询 ReceiverQueue，进行收件箱消息消费；
- **RpcEndpointRef:** RpcEndpointRef 是对远程 RpcEndpoint 的一个引用。当我们需要向一个具体的 RpcEndpoint 发送消息时，一般我们需要获取到该 RpcEndpoint 的引用，然后通过该应用发送消息。
- **OutBox:** 指令消息发件箱。对于当前 RpcEndpoint 来说，一个目标 RpcEndpoint 对应一个发件箱，如果向多个目标 RpcEndpoint 发送信息，则有多个 OutBox。当消息放入

Outbox 后，紧接着通过 TransportClient 将消息发送出去。消息放入发件箱以及发送过程是在同一个线程中进行；

- **RpcAddress**: 表示远程的 RpcEndpointRef 的地址，Host + Port。
- **TransportClient**: Netty 通信客户端，一个 OutBox 对应一个 TransportClient，TransportClient 不断轮询 OutBox，根据 OutBox 消息的 receiver 信息，请求对应的远程 TransportServer；
- **TransportServer**: Netty 通信服务端，一个 RpcEndpoint 对应一个 TransportServer，接受远程消息后调用 Dispatcher 分发消息至对应发件箱；

1.3 Spark 提交流程（YarnCluster）



第 2 章 任务的执行

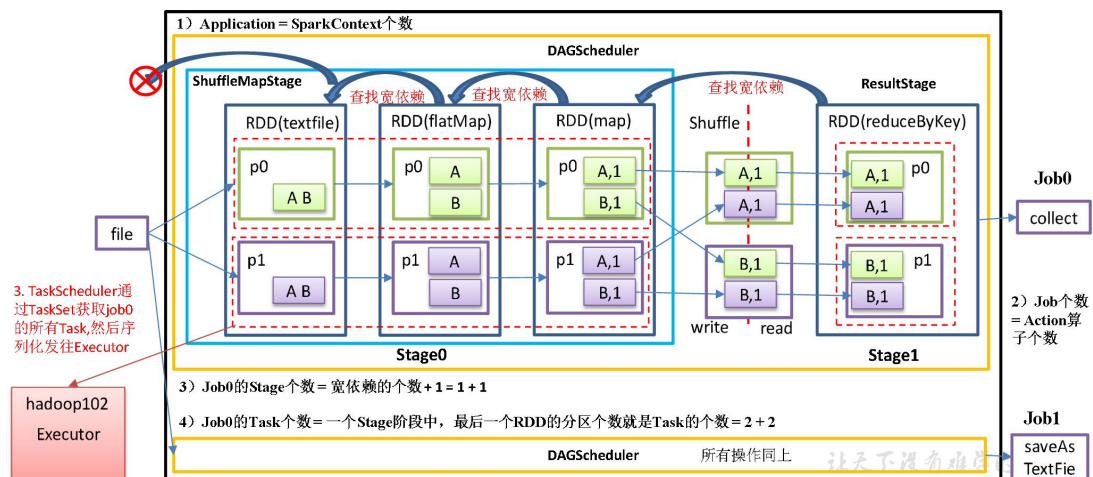
2.1 概述

2.1.1 任务切分和任务调度原理

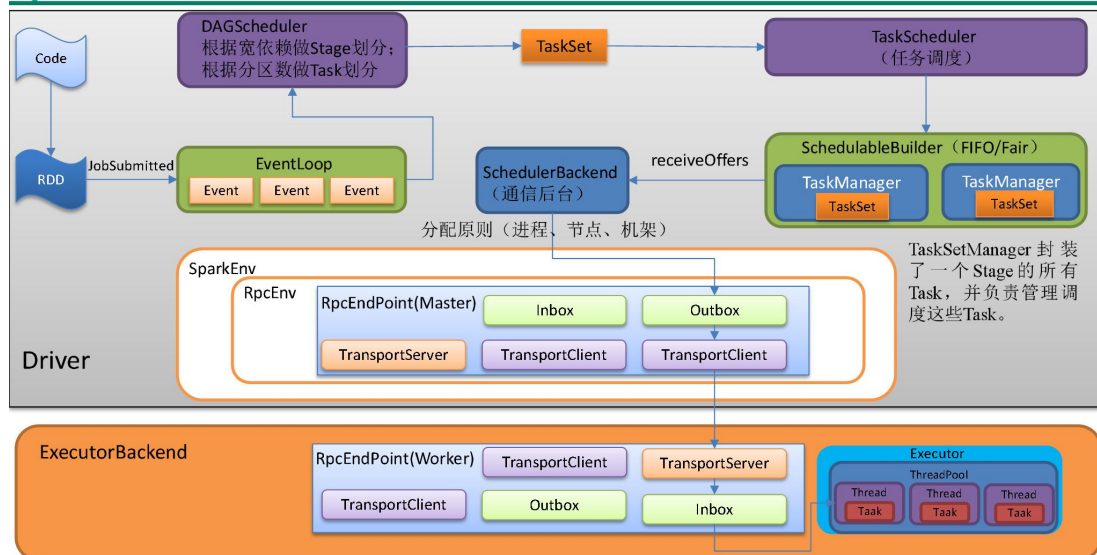


Stage任务划分

1. 执行main方法->初始化sc->执行到Action算子1
2. DAGScheduler对job0切分Stage, Stage产生Task



Task任务调度执行



2.1.2 本地化调度

任务分配原则: 根据每个 Task 的优先位置, 确定 Task 的 Locality (本地化) 级别, 本地化一共有五种, 优先级由高到低顺序:

移动数据不如移动计算。

名称	解析
PROCESS_LOCAL	进程本地化，task 和数据在同一个 Executor 中，性能最好。
NODE_LOCAL	节点本地化，task 和数据在同一个节点中，但是 task 和数据不在同一个 Executor 中，数据需要在进程间进行传输。
RACK_LOCAL	机架本地化，task 和数据在同一个机架的两个节点上，数据需要通过网络在节点之间进行传输。
NO_PREF	对于 task 来说，从哪里获取都一样，没有好坏之分。
ANY	task 和数据可以在集群的任何地方，而且不在一个机架中，性能最差。

2.1.3 失败重试与黑名单机制

Task 运行失败会告知给 TaskSetManager，对于失败的 Task，会记录它失败的次数，如果失败次数还没有超过最大重试次数，那么就把它放回待调度的 Task 池中，否则整个 Application 失败。

失败同时会记录它上一次失败所在的 Executor Id 和 Host，使用黑名单机制，避免它被调度到上一次失败的节点上，起到一定的容错作用。黑名单记录 Task 上一次失败所在的 Executor Id 和 Host，以及其对应的“拉黑”时间，“拉黑”时间是指这段时间内不要再往这个节点上调度这个 Task 了。

第 3 章 Shuffle

Spark 最初版本 HashShuffle。

Spark 0.8.1 版本以后优化后的 HashShuffle。

Spark1.1 版本加入 SortShuffle，默认是 HashShuffle。

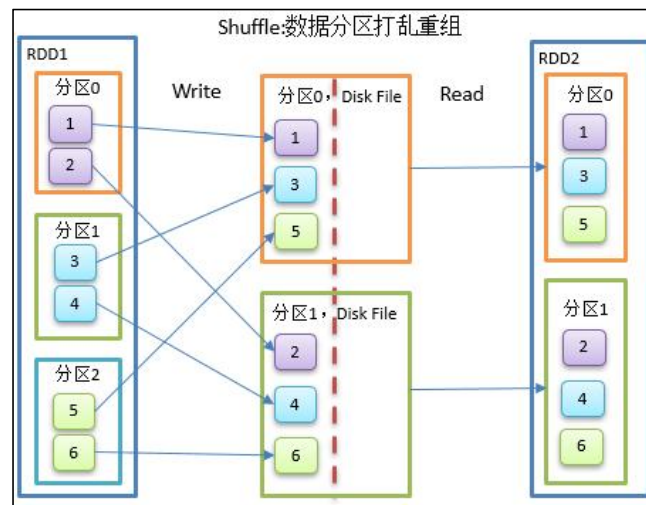
Spark1.2 版本默认是 SortShuffle，但是可配置 HashShuffle。

Spark2.0 版本删除 HashShuffle 只有 SortShuffle。

3.1 Shuffle 的原理和执行过程

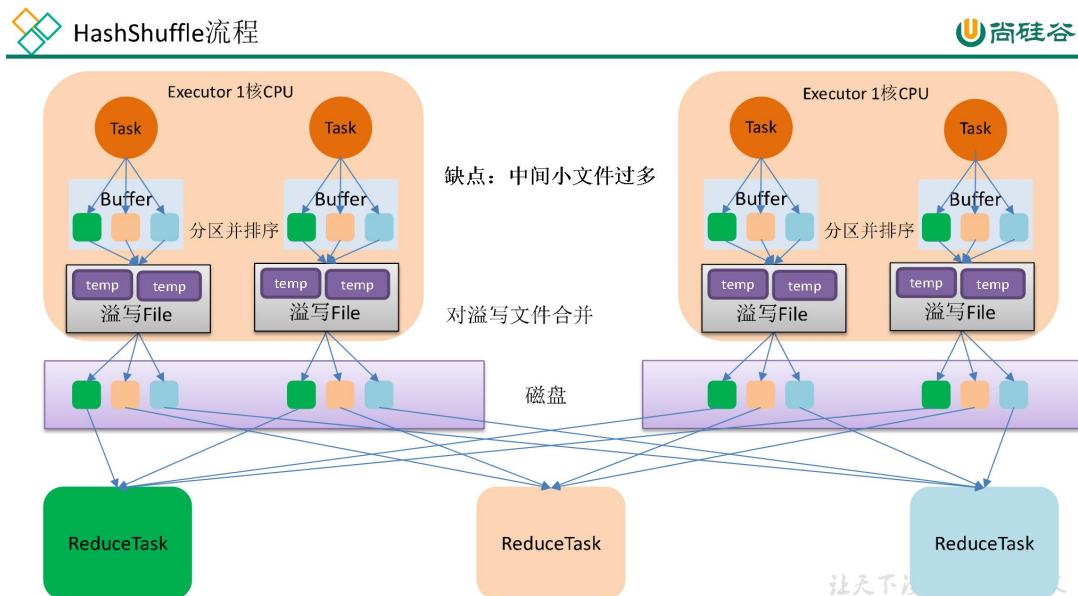
Shuffle 一定会有落盘。

- 如果 Shuffle 过程中落盘数据量减少，那么可以提高性能。
- 算子如果存在预聚合功能，可以提高 Shuffle 的性能。



3.2 HashShuffle 解析

3.2.1 未优化的 HashShuffle

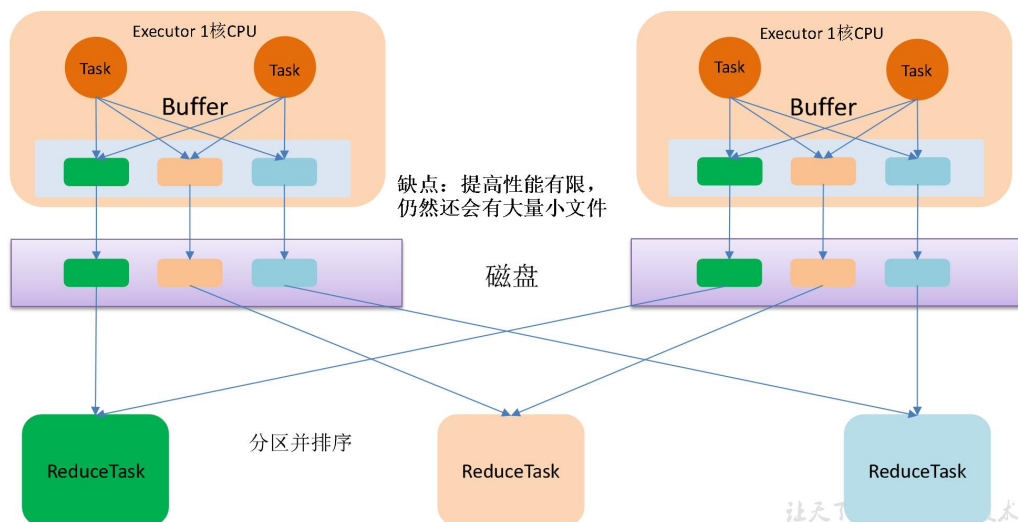


3.2.2 优化后的 HashShuffle

优化的 HashShuffle 过程就是启用合并机制，合并机制就是复用 buffer，开启合并机制的配置是 `spark.shuffle.consolidateFiles`。该参数默认值为 `false`，将其设置为 `true` 即可开启优化机制。通常来说，如果我们使用 HashShuffleManager，那么都建议开启这个选项。

官网参数说明：<http://spark.apache.org/docs/0.8.1/configuration.html>

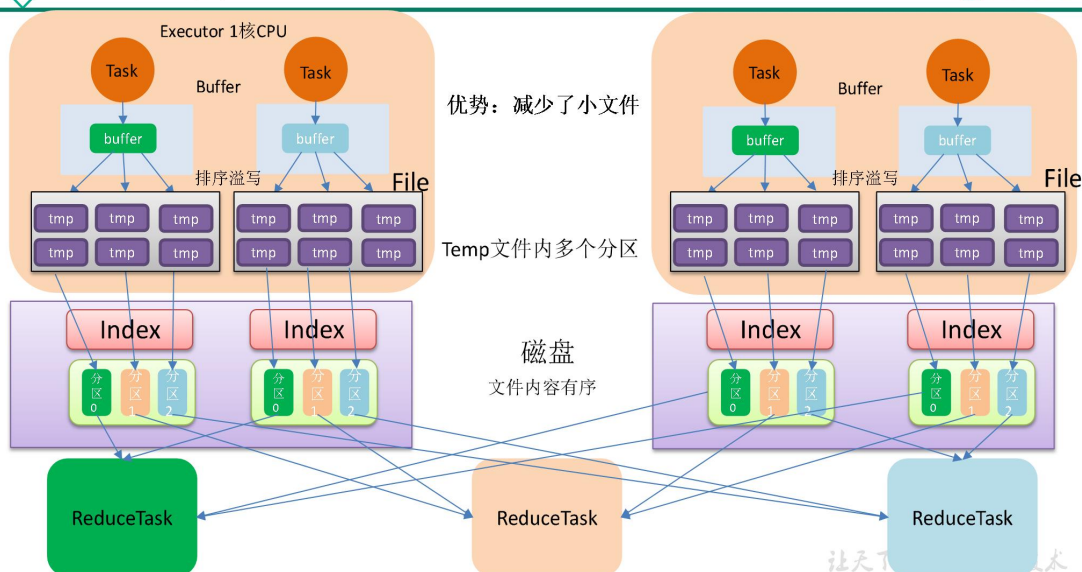
优化后的HashShuffle流程



3.3 SortShuffle 解析

3.3.1 SortShuffle

SortShuffle流程



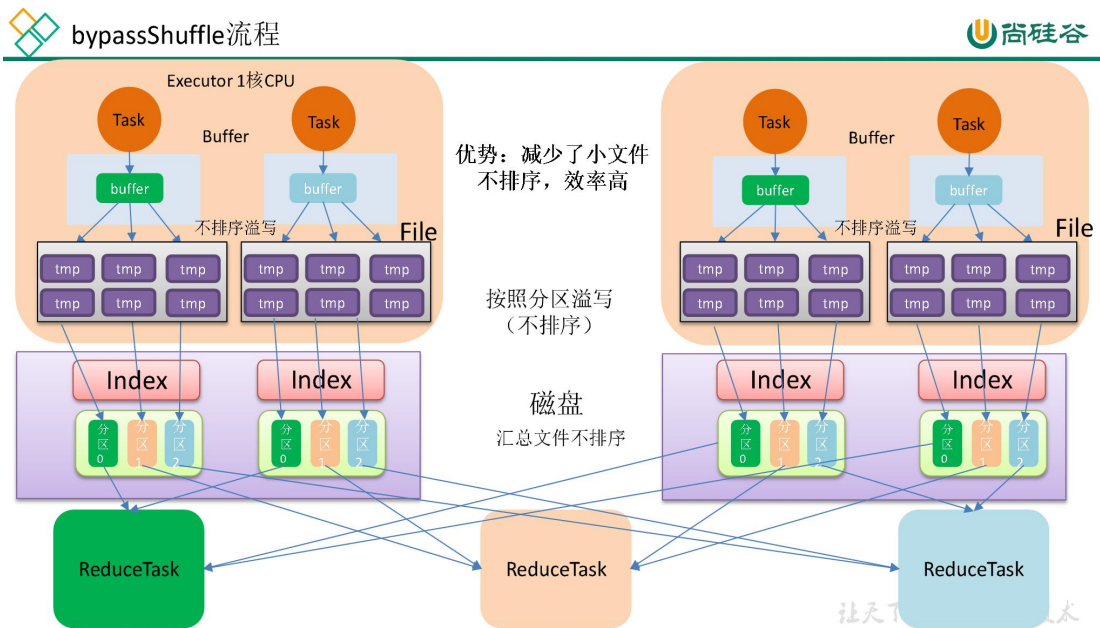
在溢写磁盘前，先根据 key 进行排序，排序过后的数据，会分批写入到磁盘文件中。默认批次为 **10000** 条，数据会以每批一万条写入到磁盘文件。写入磁盘文件通过缓冲区溢写的方式，每次溢写都会产生一个磁盘文件，**也就是说一个 Task 过程会产生多个临时文件**。最后在每个 Task 中，将所有的临时文件合并，这就是 **merge** 过程，此过程将所有临时文件读取出来，一次写入到最终文件。

3.3.2 bypassShuffle

bypassShuffle 和 SortShuffle 的区别就是不对数据排序。

bypass 运行机制的触发条件如下：

- 1) shuffle reduce task 数量小于等于 `spark.shuffle.sort.bypassMergeThreshold` 参数的值，默认为 200。
- 2) 不是聚合类的 shuffle 算子（比如 `reduceByKey` 不行）。



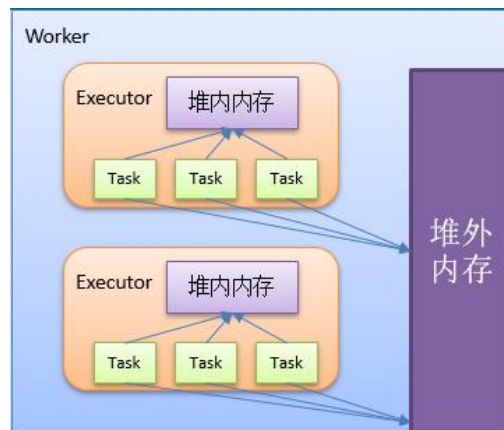
第 4 章 Spark 内存管理

4.1 堆内和堆外内存

4.1.1 概念

Spark 支持堆内内存也支持堆外内存。

- （1）堆内内存：程序在运行时动态地申请某个大小的内存空间
- （2）堆外内存：直接向操作系统进行申请的内存，不受 JVM 控制



4.1.2 堆内内存和对外内存优缺点

1) 堆外内存，相比于堆内内存有几个优势

(1) 减少了垃圾回收的工作，因为垃圾回收会暂停其他的工作

(2) 加快了复制的速度。因为堆内在 Flush 到远程时，会先序列化，然后在发送；而堆外内存本身是序列化的相当于省略掉了这个工作。

说明：堆外内存是序列化的，其占用的内存大小可直接计算。堆内内存是非序列化的对象，其占用的内存是通过周期性地采样近似估算而得。此外，在被 Spark 标记为释放的对象实例，很有可能在实际上并没有被 JVM 回收，导致实际可用的内存小于 Spark 记录的可用内存。所以 Spark 并不能准确记录实际可用的堆内内存，从而也就无法完全避免内存溢出 OOM 的异常。

2) 堆外内存，相比于堆内内存有几个缺点：

(1) 堆外内存难以控制，如果内存泄漏，那么很难排查

(2) 堆外内存相对来说，不适合存储很复杂的对象。一般简单的对象或者扁平化的比较适合。

4.1.3 如何配置

1) 堆内内存大小设置：-executor-memory 或 spark.executor.memory

2) 在默认情况下堆外内存并不启用，spark.memory.offHeap.enabled 参数启用，并由 spark.memory.offHeap.size 参数设定堆外空间的大小。

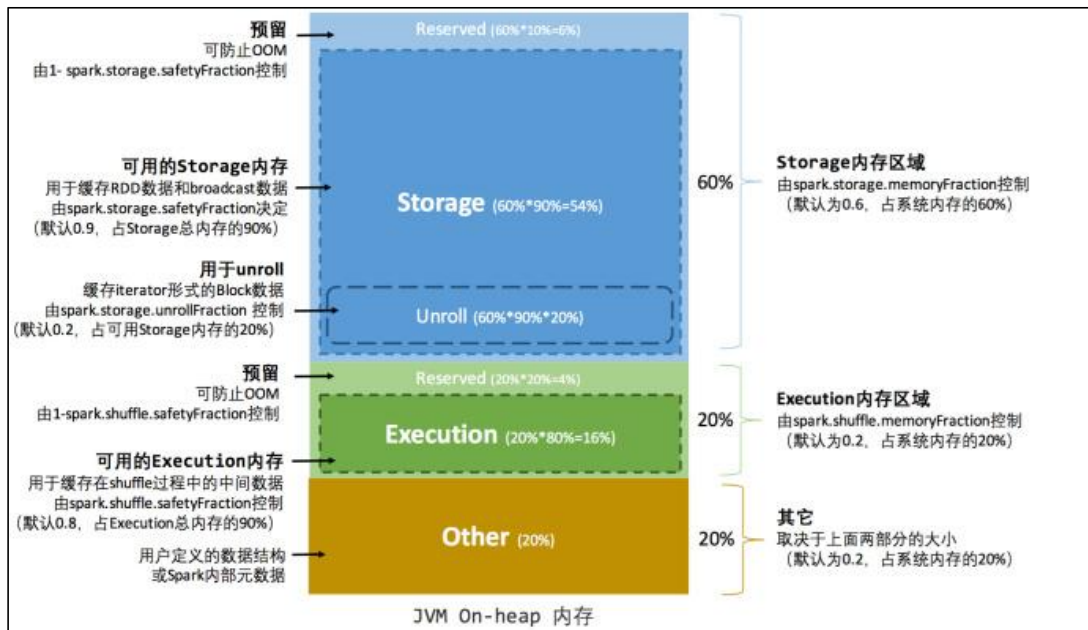
官网配置地址：<http://spark.apache.org/docs/3.3.1/configuration.html>

4.2 堆内内存空间分配

堆内内存包括：存储（Storage）内存、执行（Execution）内存、其他内存

4.2.1 静态内存管理

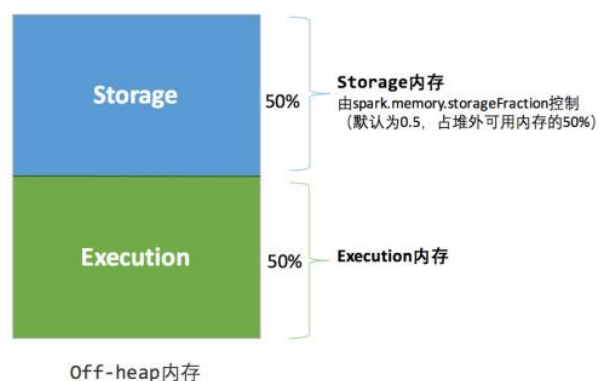
在 Spark 最初采用的静态内存管理机制下，存储内存、执行内存和其他内存的大小在 Spark 应用程序运行期间均为固定的，但用户可以应用程序启动前进行配置，堆内内存的分配如图所示：



可以看到，可用的堆内内存的大小需要按照下列方式计算：

```

可用的存储内存 = systemMaxMemory * spark.storage.memoryFraction *
spark.storage.safetyFraction
可用的执行内存 = systemMaxMemory * spark.shuffle.memoryFraction *
spark.shuffle.safetyFraction
    
```

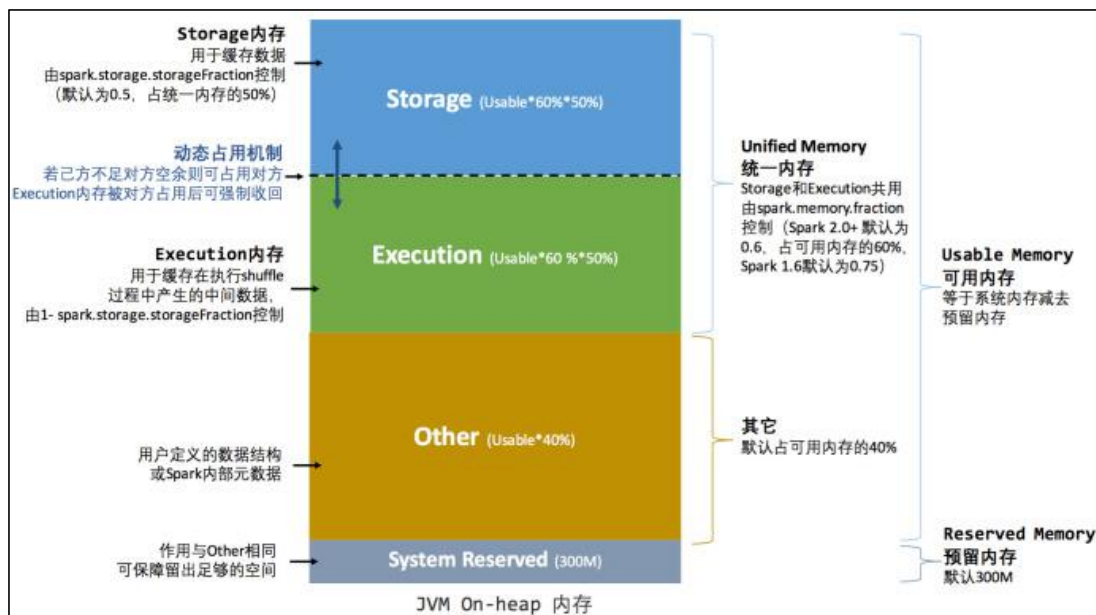


由于新的内存管理机制的出现，这种方式目前已经很少有开发者使用，出于兼容旧版本的应用程序的目的，Spark 仍然保留了它的实现。

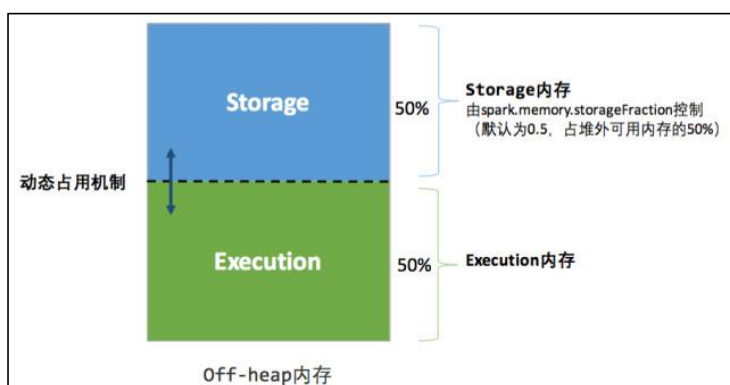
4.2.2 统一内存管理

Spark1.6 之后引入的统一内存管理机制，与静态内存管理的区别在于存储内存和执行

内存共享同一块空间，可以动态占用对方的空闲区域，统一内存管理的堆内内存结构如图所示：



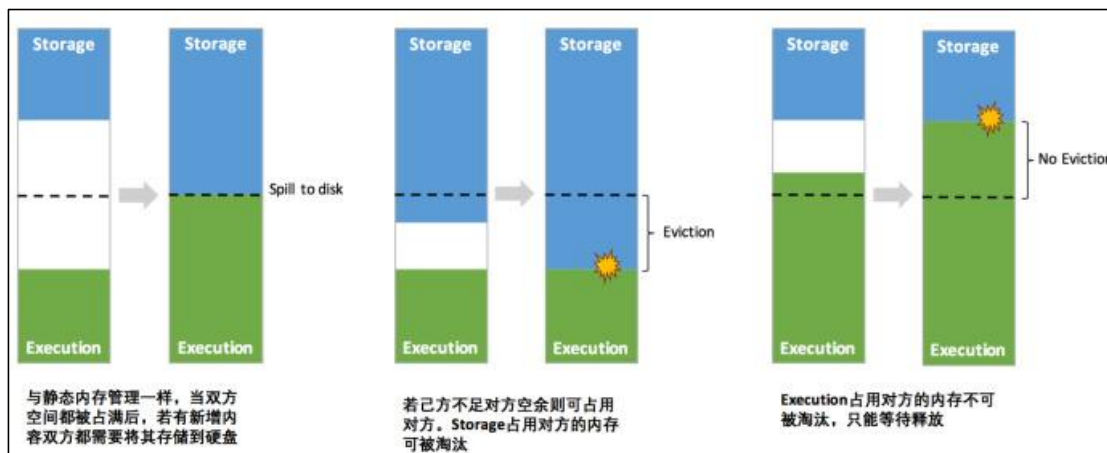
统一内存管理的堆外内存结构如下图所示：



其中最重要的优化在于动态占用机制，其规则如下：

- (1) 设定基本的存储内存和执行内存区域 (`spark.storage.storageFraction` 参数)，该设定确定了双方各自拥有的空间的范围；
- (2) 双方的空间都不足时，则存储到硬盘；若己方空间不足而对方空余时，可借用对方的空间；(存储空间不足是指不足以放下一个完整的 Block)
- (3) 执行内存的空间被对方占用后，可让对方将占用的部分转存到硬盘，然后“归还”借用的空间；
- (4) 存储内存的空间被对方占用后，无法让对方“归还”，因为需要考虑 Shuffle 过程中的很多因素，实现起来较为复杂。

统一内存管理的动态占用机制如图所示：

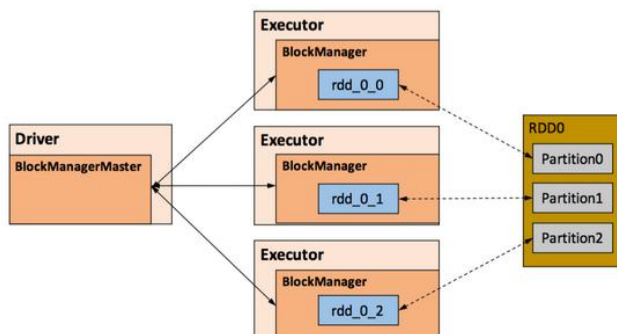


4.3 存储内存管理

4.3.1 RDD 的持久化机制

RDD 的持久化由 Spark 的 Storage 模块负责，实现了 RDD 与物理存储的解耦合。Storage 模块负责管理 Spark 在计算过程中产生的数据，将那些在内存或磁盘、在本地或远程存取数据的功能封装了起来。在具体实现时 Driver 端和 Executor 端的 Storage 模块构成了主从式的架构，即 Driver 端的 BlockManager 为 Master，Executor 端的 BlockManager 为 Slave。

Storage 模块在逻辑上以 Block 为基本存储单位，RDD 的每个 Partition 经过处理后唯一对应一个 Block（BlockId 的格式为 rdd_RDD-ID_PARTITION-ID）。Driver 端的 Master 负责整个 Spark 应用程序的 Block 的元数据信息的管理和维护，而 Executor 端的 Slave 需要将 Block 的更新等状态上报到 Master，同时接收 Master 的命令，例如新增或删除一个 RDD。



在对 RDD 持久化时，Spark 规定了 MEMORY_ONLY、MEMORY_AND_DISK 等 7 种不同的存储级别，而存储级别是以下 5 个变量的组合：

```
class StorageLevel private(
```



```
private var _useDisk: Boolean, //磁盘
private var _useMemory: Boolean, //这里其实是指堆内内存
private var _useOffHeap: Boolean, //堆外内存
private var _deserialized: Boolean, //是否为非序列化
private var _replication: Int = 1 //副本个数
)
```

Spark 中 7 种存储级别如下：

持久化级别	含义
MEMORY_ONLY	以非序列化的 Java 对象的方式持久化在 JVM 内存中。如果内存无法完全存储 RDD 所有的 partition，那么那些没有持久化的 partition 就会在下次需要使用它们的时候，重新被计算
MEMORY_AND_DISK	同上，但是当某些 partition 无法存储在内存中时，会持久化到磁盘中。下次需要使用这些 partition 时，需要从磁盘上读取
MEMORY_ONLY_SER	同 MEMORY_ONLY，但是会使用 Java 序列化方式，将 Java 对象序列化后进行持久化。可以减少内存开销，但是需要进行反序列化，因此会加大 CPU 开销
MEMORY_AND_DISK_SER	同 MEMORY_AND_DISK，但是使用序列化方式持久化 Java 对象
DISK_ONLY	使用非序列化 Java 对象的方式持久化，完全存储到磁盘上
MEMORY_ONLY_2 MEMORY_AND_DISK_2 等等	如果是尾部加了 2 的持久化级别，表示将持久化数据复用一份，保存到其他节点，从而在数据丢失时，不需要再次计算，只需要使用备份数据即可

通过对数据结构的分析，可以看出存储级别从三个维度定义了 RDD 的 Partition（同时也是 Block）的存储方式：

- **存储位置：**磁盘 / 堆内内存 / 堆外内存。如 MEMORY_AND_DISK 是同时在磁盘和堆内内存上存储，实现了冗余备份。OFF_HEAP 则是只在堆外内存存储，目前选择堆外内存时不能同时存储到其他位置。
- **存储形式：**Block 缓存到存储内存后，是否为非序列化的形式。如 MEMORY_ONLY 是非序列化方式存储，OFF_HEAP 是序列化方式存储。
- **副本数量：**大于 1 时需要远程冗余备份到其他节点。如 DISK_ONLY_2 需要远程备份 1 个副本。

4.3.2 淘汰与落盘

由于同一个 Executor 的所有的计算任务共享有限的存储内存空间，当有新的 Block 需要缓存但是剩余空间不足且无法动态占用时，就要对 LinkedHashMap 中的旧 Block 进行淘汰（Eviction），而被淘汰的 Block 如果其存储级别中同时包含存储到磁盘的要求，则要对其进行落盘（Drop），否则直接删除该 Block。

存储内存的淘汰规则为：

- 被淘汰的旧 Block 要与新 Block 的 MemoryMode 相同，即同属于堆外或堆内内存；
- 新旧 Block 不能属于同一个 RDD，避免循环淘汰；
- 旧 Block 所属 RDD 不能处于被读状态，避免引发一致性问题；
- 遍历 LinkedHashMap 中 Block，按照最近最少使用（LRU）的顺序淘汰，直到满足新 Block 所需的空间。其中 LRU 是 LinkedHashMap 的特性。

落盘的流程则比较简单，如果其存储级别符合 `_useDisk` 为 `true` 的条件，再根据其 `_deserialized` 判断是否是非序列化的形式，若是则对其进行序列化，最后将数据存储到磁盘，在 `Storage` 模块中更新其信息。

4.4 执行内存管理

执行内存主要用来存储任务在执行 Shuffle 时占用的内存，Shuffle 是按照一定规则对 RDD 数据重新分区的过程，我们来看 Shuffle 的 Write 和 Read 两阶段对执行内存的使用：

1) Shuffle Write

若在 map 端选择快速排序方式，在内存中存储数据时主要占用堆内执行空间。

在内存中存储数据时可以占用堆外或堆内执行空间，取决于用户是否开启了堆外内存以及堆外执行内存是否足够。

2) Shuffle Read

在对 reduce 端的数据进行聚合时，要将数据交给 Aggregator 处理，在内存中存储数据时占用堆内执行空间。